Candle chart vs Lightweight Charts

8 of 13
features do the same work as LWC

4
features do extra work: live tick, live append, axis labels and drawing drags

1
feature is close enough: autoscale has no chunk cache

Feature by feature

Feature	What LWC does	What this chart does	Extra work vs LWC	Result
Architecture	Plot list holds the data; `TimeScale` and `PriceScale` hold the view	`ChartBloc` holds data only; `TimeScale`, `PriceScale` and `DrawingController` are `ChangeNotifier`s	**None**	✅ **Same split**
Drawing model	Main and top canvas per pane, plus axis canvases	Pane painter, drawings layer, crosshair layer and 3 axis painters, each in its own `RepaintBoundary` (7 `CustomPaint`s)	**None**	✅ **Same shape**
Tree size	Fixed number of canvases	Fixed: 200 elements and 68 render objects at any zoom, scroll or number of drawings	Constant Flutter overhead; doesn't grow	✅ **Same**
First render	1 frame, every canvas drawn	2.1 ms; 154 rebuilds (mostly `MaterialApp` setup), 67 layouts, 7 `CustomPaint`s	+154 rebuilds and +67 layouts, **once** at startup	✅ **Same, plus Flutter's one-off setup**
Pan and fling	"Light" update: change `rightOffset`, redraw pane and axes	0.33–0.35 ms per frame; 0 rebuilds, 0 layouts, 0 bloc events; 6 `CustomPaint`s repainted	**None** (0 rebuilds, 0 layouts, 0 bloc events)	✅ **Same**

Pinch zoom	"Light" update: change `barSpacing`, redraw pane and axes	0.38 ms per step; 0 rebuilds, 0 layouts; 6 `CustomPaint`s repainted	**None** (0 rebuilds, 0 layouts)	✅ **Same**
Crosshair	"Cursor" update: redraw the top canvas only	0.17 ms per move; 1 `CustomPaint` and 2 render objects repainted	**None** (1 layer, like LWC)	✅ **Same**
Price drag and axis scale	Change the price scale, redraw pane and price axis	`PriceScale` notifies; pane and price axis repaint, nothing rebuilds	**None** by design (not benchmarked)	✅ **Same**
Live tick	Updates the last plot row in place, then a "Light" redraw; 0 rebuilds	0.75 ms; copies the 1,000-candle list, then 19 rebuilds and 8 layouts, then 6 `CustomPaint`s. With 10 drawings: the same counts	**+19 rebuilds, +8 layouts, 1,000 candles copied instead of 1 row.** About **+0.40 ms**, so a tick costs **2.1×** a paint-only frame	⚠️ **Extra work**
Live append	Adds a row; the view shifts if following live	0.65–0.66 ms; same as a tick, plus `TimeScale.candlesAppended`	**Same as a tick.** About **+0.31 ms**, so an append costs **1.9×** a paint-only frame	⚠️ **Extra work**
Autoscale (and double-tap reset)	Lazy, cached per chunk of bars	Lazy; cached by candle list and visible range	One loop over 50–200 visible bars on each cache miss; LWC reuses cached chunk min/max	**≈ Fine at up to 200 bars**
Axis labels	`TextWidthCache` reuses measured labels	A new `TextPainter` for every label on every repaint	**About 20–25 label layouts per pan or zoom frame**, where LWC reuses cached ones	⚠️ **No cache**
Pixel snapping	Device-pixel bitmap space, `optimalCandlestickWidth`, wick parity	Ported: `optimalCandleWidth` with device-pixel snapping and parity	**None**	✅ **Same**

Draw or move a line	No built-in tools; lines are plugins (series primitives). A change calls `requestUpdate`, which is a `fullUpdate()`: sizes are recomputed and every canvas redraws, candles included	0.09–0.11 ms per drag step; only the drawings layer repaints (1 `CustomPaint`); 3 rebuilds and 8 layouts	**+3 rebuilds and +8 layouts per step.** Paints **less** than LWC: the candles stay cached	⚠️ **Extra tree work**
Pan with 10 drawings	Primitives redraw with the pane and axes on every "Light" update	0.39–0.40 ms per frame; 0 rebuilds, 0 layouts; 6 `CustomPaint`s, the same as with no drawings	**None.** 10 drawings add about **+0.05 ms** of paint per frame	✅ **Same**
Frame scheduling	Merges all changes into one `requestAnimationFrame`	Flutter merges `markNeedsPaint` calls into one frame	**None**	✅ **Same**

Drawing tools

Horizontal and trend lines, measured with 1,000 candles, 50 visible, at 800 × 500. Each scenario runs 60 frames, and the numbers are per frame.

Scenario	Frame time	Rebuilds	Layouts	`CustomPaint`s repainted	Bloc events	What it means
Drag out a trend line	0.11 ms	3	8	1	0	Only the drawings layer repaints; the candles stay cached
Move a trend line	0.09 ms	3	8	1	0	Same as dragging one out
Pan, no drawings	0.33–0.35 ms	0	0	6	0	Baseline
Pan, 10 drawings	0.39–0.40 ms	0	0	6	0	About +0.05 ms for 10 lines; no tree work
Live tick, no drawings	0.75 ms	19	8	6	1	Baseline
Live tick, 10 drawings	0.63–0.68 ms	19	8	6	1	The same counts; the time difference is run-to-run noise

Where it is better than LWC

- **Drawing drags repaint 1 layer, not the pane.** The drawings have their own layer, so dragging a line leaves the candles, axes and crosshair cached. In LWC, a primitive's `requestUpdate` triggers a `fullUpdate()`, which redraws every canvas (`series-api.ts`). A drag step costs about **a third of a pan frame** (0.09–0.11 ms against 0.35 ms).
- **It's the only place where it does less painting than LWC.** The 3 rebuilds per step cancel part of that gain; see the fixes below.

Where it matches

- **Interaction is paint-only.** Pan, fling, zoom, crosshair and price drag change a few numbers on the scales, and the painters redraw. Nothing rebuilds, lays out or goes through the bloc, with or without drawings.
- **The tree doesn't grow.** It stays at 200 elements whatever the zoom, the scroll or the number of drawings.
- **Candles are pixel-sharp,** using the same width and snapping rules as LWC.

Where it still does more, and the fix

1. **Live ticks: +19 rebuilds and +8 layouts per tick (LWC: 0).** `CandleChart` and `DrawingsLayer` both select `candles`, so a new list rebuilds them. Fix: pass candles to the painters through a notifier so a tick only repaints. Expected: a tick costs close to a paint-only frame.
2. **Each tick copies 1,000 candles (LWC: 1 row).** The copy grows with the history loaded. Fix: a mutable buffer plus a revision counter, so a tick costs the same at any history size.
3. **About 20–25 axis labels are laid out per pan or zoom frame (LWC: cached).** Fix: cache laid-out labels by their text, as LWC's `TextWidthCache` does.
4. **Drawing drags: +3 rebuilds and +8 layouts per step.** The `ListenableBuilder` around the `GestureDetector` listens to the whole `DrawingController`, so every drag step rebuilds it. It only needs to rebuild when the tool changes, to turn double-tap on or off. Fix: listen to the tool alone, for example through a separate `ValueNotifier<DrawingTool?>`. A drag step would then be paint-only.